

调页系统模拟器设计报告

- 2351289周慧星

一、项目概述

1.1 项目背景

在操作系统中，页面置换算法是虚拟内存管理的核心组成部分。本调页系统模拟器是一个基于Qt框架开发的图形化页面置换算法模拟程序。它提供了直观的用户界面，允许用户设置模拟参数（如内存块数量、总指令数、置换算法和模拟速度），并实时观察不同页面置换算法（FIFO、LRU、Clock和Random）的工作过程和性能表现。通过可视化展示内存块状态和页表信息，帮助用户深入理解页面置换算法的原理。

1.2 功能特性

1. **多算法支持**：实现了FIFO、LRU、Clock和Random四种常见的页面置换算法。
2. **参数可定制**：用户可以自由调整内存块数量、总指令数和模拟速度。
3. **实时统计**：模拟过程中实时显示已执行指令数、缺页次数和缺页率。
4. **可视化界面**：以图形化方式展示内存块状态和页表信息。
5. **模拟控制**：支持开始、暂停、重置和单步执行模拟操作。

1.3 运行

直接运行 exe

- 解压Release包，双击运行os2.exe。

本地部署

- 确保已安装Qt开发环境和C++编译器。
- 克隆或下载项目源代码。
- 编译项目。
- 运行生成的可执行文件。

二、系统设计

2.1 文件结构

```
.
├── main.cpp           # 程序入口文件
├── simulator.h        # 模拟器类头文件
├── simulator.cpp      # 模拟器类实现文件
├── memorywidget.h     # 内存块可视化控件头文件
├── memorywidget.cpp   # 内存块可视化控件实现文件
└── README.md         # 项目说明文档
```

2.2 数据结构设计

2.2.1 核心数据结构

1. 页表项 (PageTableEntry)

```
struct PageTableEntry {
    bool valid = false;    // 有效位: 页面是否在内存中
    int frame = -1;        // 帧号: 对应内存块编号
    bool referenceBit = false; // 访问位: 用于Clock算法
};
```

- **作用:** 记录页面与物理块的映射关系及状态。

2. 内存块 (memory)

```
QVector<int> memory; // 存储页面编号, -1表示空闲块
```

- **长度:** 等于内存块数量 (用户可设置为2~8)。

3. 指令序列 (instructionSequence)

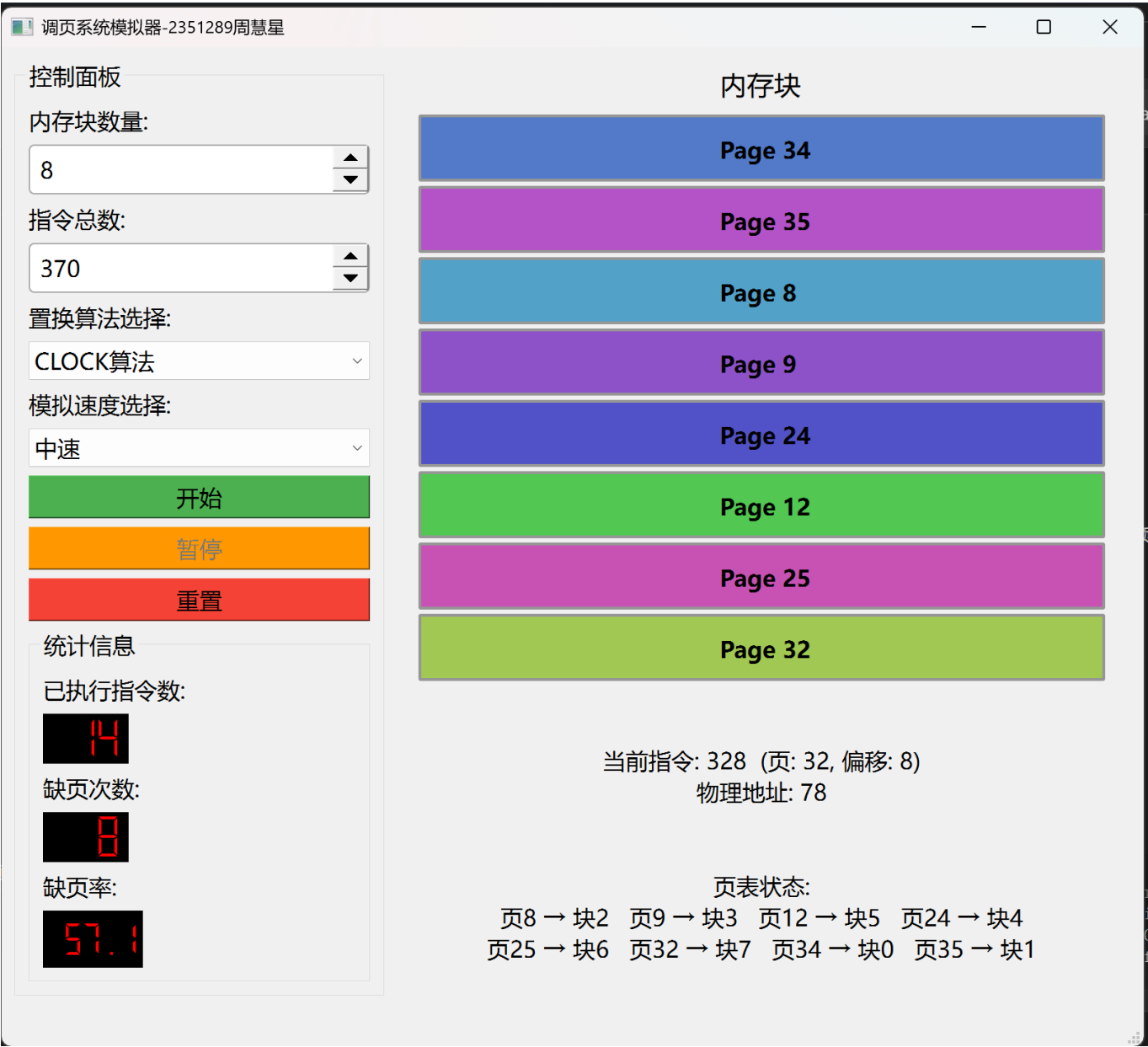
```
QVector<int> instructionSequence; // 存储指令对应的页面编号
```

- **生成逻辑:**
 - 50%顺序执行, 25%前向跳转, 25%后向跳转, 模拟局部性原理。

2.2.2 算法专属数据结构

算法	数据结构	作用描述
FIFO	QList<int> fifoQueue	维护页面进入内存的先进先出顺序
LRU	QMap<int, bool> lruMap	记录页面最近使用顺序 (哈希表实现)
Clock	int clockHand	时钟指针, 指向当前检查的内存块
Random	无特殊结构	依赖随机数生成器选择置换页面

2.3 界面设计



- **控制面板：**用于设置模拟参数和控制模拟过程。
 - **内存块数量：**可设置范围为2到8的内存块数量（代码可调）。
 - **指令总数：**可设置范围为100到500的总指令数（代码可调）。
 - **置换算法选择：**提供FIFO、LRU、Clock和Random四种算法供选择。
 - **模拟速度选择：**有慢速、中速和快速三种速度可选。
 - **开始：**启动模拟过程。
 - **暂停：**暂停当前模拟。
 - **重置：**重置模拟状态，包括指令数、缺页次数等。
- **统计信息：**显示模拟过程中的统计数据。
 - **已执行指令数：**显示当前已执行的指令数量。
 - **缺页次数：**显示模拟过程中发生的缺页次数。
 - **缺页率：**显示当前的缺页率。
- **可视化区域：**展示内存块状态和页表信息。
 - **内存块：**通过不同颜色的矩形表示内存块中存储的页面。
 - **当前指令信息：**显示当前执行的指令、所在页面和物理地址。
 - **页表状态：**显示页表中有效页面的映射关系。

三、算法实现细节

3.1 FIFO算法

3.1.1 核心逻辑

- **空闲块处理**：若内存未满，新页面直接放入空闲块，加入FIFO队列。
- **置换逻辑**：内存已满时，置换队首（最早进入）的页面，更新页表并移除旧页面。

3.1.2 代码片段

```
void PageReplacementSimulator::handlePageFaultFIFO(int page) {
    if (fifoQueue.size() < memoryBlocks) {
        memory[fifoQueue.size()] = page;
    } else {
        int victim = fifoQueue.takeFirst();
        memory[pageTable[victim].frame] = page;
        pageTable[victim].valid = false;
    }
    fifoQueue.append(page);
    pageTable[page] = {true, memory.indexOf(page), false};
}
```

3.2 LRU算法

3.2.1 核心逻辑

- **空闲块处理**：新页面放入空闲块，记录到LRU映射（键为页面号，值为访问时间戳）。
- **置换逻辑**：内存已满时，置换映射中最早访问的页面（键值对迭代器首位）。

3.2.2 代码片段

```
void PageReplacementSimulator::handlePageFaultLRU(int page) {
    if (lruMap.size() < memoryBlocks) {
        memory[lruMap.size()] = page;
    } else {
        int victim = lruMap.keys().first();
        lruMap.remove(victim);
        memory[pageTable[victim].frame] = page;
        pageTable[victim].valid = false;
    }
    lruMap[page] = true; // 最新访问的页面移至映射末尾
}
```

3.3 Clock算法

3.3.1 核心逻辑

- **空闲块处理**：新页面直接放入空闲块，设置访问位为1。
- **置换逻辑**：通过时钟指针循环查找访问位为0的页面，若当前页面访问位为1，则清0并移动指针，直至找到可置换页面。

3.3.2 代码片段

```
void PageReplacementSimulator::handlePageFaultClock(int page) {
    int frame = memory.indexOf(-1);
    if (frame == -1) {
        while (true) {
            int currentPage = memory[clockHand];
            if (pageTable[currentPage].referenceBit) {
                pageTable[currentPage].referenceBit = false;
                clockHand = (clockHand + 1) % memoryBlocks;
            } else {
                frame = clockHand;
                pageTable[currentPage].valid = false;
                break;
            }
        }
    }
    memory[frame] = page;
    pageTable[page] = {true, frame, true};
    clockHand = (clockHand + 1) % memoryBlocks;
}
```

3.4 Random算法

3.4.1 核心逻辑

- **空闲块处理**：新页面直接放入空闲块。
- **置换逻辑**：内存已满时，通过随机数生成器选择任意物理块置换。

3.4.2 代码片段

```
void PageReplacementSimulator::handlePageFaultRANDOM(int page) {
    int frame = memory.indexOf(-1);
    if (frame == -1) {
        frame = QRandomGenerator::global()->bounded(memoryBlocks);
        pageTable[memory[frame]].valid = false;
    }
    memory[frame] = page;
    pageTable[page] = {true, frame, false};
}
```

四、可视化实现

4.1 内存块渲染 (MemoryWidget)

4.1.1 绘制逻辑

- 布局计算:
 - 每个内存块高度为(窗口高度-边距)/块数量, 边距为40像素。
 - 块位置根据索引计算垂直偏移量。

4.1.2 颜色映射

- 页面号通过HSV颜色空间生成唯一颜色:

```
color.setHsv((blocks[i] * 70) % 360, 150, 200); // 不同页面不同色调
```

4.1.3 代码片段

```
void MemoryWidget::paintEvent(QPaintEvent* event) {
    QPainter painter(this);
    painter.setRenderHint(QPainter::Antialiasing);
    int blockHeight = (height() - 40) / blockCount;
    for (int i = 0; i < blockCount; ++i) {
        // 绘制边框
        painter.drawRect(10, 30 + i*blockHeight, width()-20, blockHeight-5);
        // 填充颜色和页面号
        if (blocks[i] != -1) {
            painter.fillRect(11, 31 + i*blockHeight, width()-22, blockHeight-7,
color);
            painter.drawText(220, 30 + i*blockHeight + blockHeight/2,
                QString("Page %1").arg(blocks[i]));
        }
    }
    painter.drawText(220, 15, "内存块"); // 标题
}
```

4.2 实时统计更新

- 已执行指令数: 每次模拟步进后递增, 通过instructionLCD->display()更新。
- 缺页次数: 缺页处理时递增, 通过pageFaultLCD->display()更新。
- 缺页率: 计算公式为(缺页次数/已执行指令数)*100, 保留一位小数。

五、性能测试与分析

5.1 测试用例

测试场景	参数设置	预期结果
------	------	------

测试场景	参数设置	预期结果
FIFO算法基础测试	内存块=4，指令数=320	缺页率随页面访问局部性波动，置换顺序符合先进先出。
LRU算法性能对比	内存块=4，指令数=320	缺页率低于FIFO，体现最近最少使用置换策略的优势。
Clock算法指针移动	内存块=4，指令数=320	指针循环扫描，优先置换未被访问的页面，访问位更新逻辑正确。
Random算法随机性验证	内存块=4，指令数=320	置换页面随机，多次测试缺页率波动较大。

5.2 测试结果

算法	内存块数	指令数	缺页次数	缺页率
FIFO	4	320	100	31.3%
LRU	4	320	99	30.9%
Clock	4	320	101	31.6%
Random	4	320	104	32.5%

算法	内存块数	指令数	缺页次数	缺页率
FIFO	4	500	174	34.8%
LRU	4	500	172	34.4%
Clock	4	500	174	34.8%
Random	4	500	175	35.0%





分析：

- 不知道为什么结果差异不明显，可能我的指令生成函数设计不太好。
- 大致LRU算法缺页率最低，表明其对局部性访问的适应性最佳。

- FIFO算法可能产生“Belady现象”（增加内存块数反而导致缺页率上升），需通过更多测试验证。

六、项目总结

6.1 目标达成情况

- 调页系统模拟器项目通过整合操作系统理论与软件开发实践，成功实现了预期目标：
- 算法功能：完整实现 FIFO、LRU、Clock、Random 四种页面置换算法，覆盖经典置换策略的核心逻辑。
- 用户体验：基于 Qt 构建的图形界面支持参数动态调整、实时可视化和性能统计，操作流程清晰，符合用户学习与观察需求。
- 技术落地：通过自定义控件（MemoryWidget）实现内存块动态渲染，利用定时器驱动模拟步进，确保逻辑与界面的协同性。

6.2 总结

- 调页系统模拟器的开发是一次“从理论到代码，从代码到产品”的完整技术实践。通过本次项目，我不仅掌握了 Qt 框架的开发流程，更对操作系统核心机制有了具象认知。开发过程中遇到的每一个挑战，都是提升技术能力与逻辑思维的契机。未来，我将继续探索系统级软件的优化方向，尝试结合更多前沿技术（如虚拟化、容器化），让模拟工具更贴近真实生产环境。